CS260 Reading Assignment, Week 3

Question 1: Notes on the Reading

12.1 Supported Operations

- hash table = a dictionary
- 3 main ops: Lookup (find by key), Insert (add), Delete (remove by key)
- all 3 ops typically O(1), but only if the table/hash function are set up right and the data isn't pathological
- space used is proportional to the number of objects actually stored, way better than sizing an array for every possible key
- rule of thumb: fast lookups + a set of objects that keeps changing = use a hash table

12.2 Applications

- de-duplication: for each new key, Lookup first, Insert if not found, ends up with one entry per distinct key
- symbol tables in compilers (tracking variable/function names), basically why hash tables were invented
- 2-SUM problem: brute force is O(n^2), but throwing everything into a hash table and checking for (t - x) drops it to O(n)
- basically: anytime there's repeated lookups going on, a hash table should come to mind

12.3 Implementation: High-Level Ideas

- universe U = all possible keys (huge), set S = keys actually stored (small compared to U)
- big array sized to U = fast but wastes tons of space
- linked list = good space but slow O(|S|) lookups
- hash table = best of both, O(|S|) space and roughly O(1) ops
- hash function h maps every key in U to a position 0 to n-1 in the array
- collisions: two different keys hashing to the same spot, unavoidable once |U| is bigger than n (pigeonhole principle)
- birthday paradox makes it worse: collisions show up way sooner than expected even in a mostly-empty table
- collision resolution method 1, chaining: each bucket holds a linked list, insert is O(1), lookup/delete depends on bucket length
- collision resolution method 2, open addressing: each slot holds 0 or 1 objects, each key has a probe sequence of positions to try in order
- linear probing: try h(k), h(k)+1, h(k)+2, etc, wrapping around
- double hashing: one hash function picks the start position, a second picks the step size between tries
- good hash function: cheap to compute, cheap to store, spreads normal data evenly across slots
- bad example: h(k) = 0 for everything, turns the whole table into one giant linked list
- pathological data sets exist for every hash function, so O(1) can only be guaranteed for non-pathological/realistic data (that's what the asterisks in the book mean)

Question 2

(a) What makes a hash function “good”?

- spreads keys around evenly and unpredictably so no bucket gets overloaded
- cheap to compute and cheap to store
- keeps lookup, insert, and delete close to O(1) for normal, non-pathological data

(b) What are collisions?

- when two different keys hash to the same slot/position in the table

(c) Two primary collision resolution methods

- Chaining: every bucket holds a linked list, colliding keys just get added to that bucket's list
- Open addressing: no lists, each key has a probe sequence of positions to try in order until it finds an empty one (or finds the key, for lookup)

(d) Universe U = {xy | x in {a,b,c,d,e}, y in {1,2,3,4,5}}

- |U| = 5 choices for x times 5 choices for y = 25 = n
- sqrt(n) = 5
- hash function: h(xy) = index of x (a=0, b=1, c=2, d=3, e=4)
- uses exactly 5 slots, each slot holds at most 5 keys (one per value of y, like slot 0 holding a1 through a5)
- since 5 = sqrt(25) = sqrt(n), this satisfies the sqrt(n) slot bound